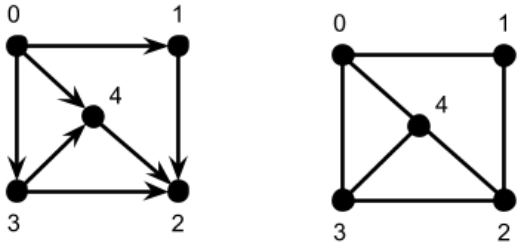
M2201 - Introduction to Graph Theory

Adjacency matrix and basic algorithms

TD3

1 Adjacency Matrix.

Give the adjacency matrices M1 and M2 of the following graphs:



2 Powers of an adjacency matrix.

→ Multiply each matrix by itself. In other words, compute $M1^2 = M1 \times M1$ and $M2^2 = M2 \times M2$.

→ What is the value of these elements:
$M1^2[0,0]$? $M1^2[0,2]$? $M1^2[0,4]$? $M1^2[2,0]$?
$M2^2[0,0]$? $M2^2[0,2]$? $M2^2[0,4]$? $M2^2[2,0]$?

→ Can you find a natural property of the graph to which these numbers correspond?

→ What do you conclude?

3 Degree of a vertex.

Write a function that finds the degree of a vertex v (given as an integer) in an undirected graph G (given as an adjacency matrix). The signature is given.

```
Function getDegree (E G : Matrix, E n :  Integer, E v :  Integer ) :  Integer
```

4 In-degree and out-degree in a *directed* graph.

- What would you change to return the out-degree of the given vertex ?
- Same question for the in-degree.

5 Maximum degree.

Write a function that computes the maximum degree $\Delta(G)$ of an undirected graph G. (You can reuse previously defined functions.)

```
Function maxDegree (E G : Matrix, E n :  Integer ) :  Integer
```

6 Complete graphs.

Write a function that tests if the given graph is complete.

```
Function isComplete (E G : Matrix, E n :  Integer ) :  Boolean
```

7 Tournament.

What strategy would you use to test if the given directed graph is a tournament.

8 Regular graphs.

Write a function that returns k if the given graph is k-regular (and -1 otherwise). Again, you can reuse existing functions...

```
Function isRegular (E G : Matrix, E n :  Integer ) :  Integer
```

9 Eulerian graphs.

Write a function that tests if the given graph contains an *eulerian* cycle (*i.e.* it is possible to cross every edge exactly once and return to the initial position).

```
Function hasEulerianCycle (E G : Matrix, E n :  Integer ) :  Boolean
```

Same question for eulerian paths.

```
Function hasEulerianPath (E G : Matrix, E n :  Integer ) :  Boolean
```

10 Connected graphs.

Write a function that tests if the given graph is connected.

```
Function isConnected (E G : Matrix, E n :  Integer ) :  Boolean
```

11 Trees.

Write a function that tests if the given graph is a tree. First decide on a strategy; testing for the existence of cycles may be too complicated... there is a better approach.